

Recipe Vibe App - Complete Project Scope Document

Executive Summary

The Recipe Vibe App is an AI-powered recipe discovery platform that matches user moods/vibes with recipes. Users describe their vibe ("cozy comfort food," "energetic breakfast," "date night romantic"), and the app uses RAG (Retrieval-Augmented Generation) to find perfect recipes with personalized explanations.

Technology Stack: Completely free, running locally on macOS with VS Code.

Table of Contents

1. [Project Overview](#)
 2. [Technology Stack](#)
 3. [Data Sources & Pipeline](#)
 4. [System Architecture](#)
 5. [Database Schema](#)
 6. [Project Structure](#)
 7. [Feature Specifications](#)
 8. [Implementation Phases](#)
 9. [Cost Analysis](#)
 10. [Deployment Guide](#)
-

Project Overview

Problem Statement

Users struggle to decide what to cook based on their mood, cravings, and available ingredients. Traditional recipe apps use keyword search, not semantic understanding of emotions and vibes.

Solution

Build a vibe-based recipe recommendation engine that understands emotional context and suggests recipes through conversational AI.

Key Features

MVP Features (Phase 1)

- **Vibe-Based Search:** Describe mood → Get personalized recipes
- **Inventory Matching:** Input available ingredients → Filter recipes
- **Combined Search:** Vibe + inventory + time constraints
- **Recipe Details:** Name, ingredients, instructions, cook time
- **RAG Explanations:** Why each recipe matches your vibe

Future Features (Phase 2+)

- Meal planning & shopping lists
- User preferences & favorites
- Seasonal recommendations
- Social vibe matching
- Nutritional tracking
- Cooking difficulty progression
- Cuisine fusion explorer

Target Users

- Home cooks (20-45 years old)
 - People experiencing decision paralysis while cooking
 - Health-conscious individuals
 - Adventure seekers exploring new cuisines
-

Technology Stack

Frontend

```
Framework:    React 18 (with Vite)
Language:     JavaScript/TypeScript
Styling:      Tailwind CSS
Components:   DaisyUI
Build Tool:   Vite
```

Hosting: Vercel (Free tier)
Package Manager: npm

Why React + Vite:

- Vite = faster development & builds than Create React App
- React ecosystem mature & robust
- DaisyUI = beautiful pre-built components for food app
- Tailwind = rapid UI development

Backend

Language: Python 3.11+
Framework: FastAPI
Server: Uvicorn
Hosting: Railway (Free \$5/mo credit)
Package Manager: pip

Why FastAPI:

- Built for AI/ML integration
- Async support (non-blocking requests)
- Auto-generated API documentation
- Excellent performance
- Perfect for RAG pipelines

Vector Database & Storage

Primary DB: Supabase (PostgreSQL + pgvector)
Vector Storage: pgvector extension
Free Tier: 500MB storage
Capacity: ~2.5M recipes indexed

Why Supabase:

- Free pgvector extension for vector search
- Built-in PostgreSQL for metadata
- Same database handles both vectors & structured data
- Easy Python integration via supabase-py SDK

Embeddings Model (Local)

Model:	BAAI/bge-small-en-v1.5
Library:	Sentence-Transformers
Dimensions:	384
Running On:	Local machine (GPU optional)
Cost:	\$0 (open-source)
Speed:	~5-10ms per embedding

Why Local Embeddings:

- Completely free (no API calls)
- No rate limits
- Fast inference
- Privacy-first (data stays local)
- Good quality for food domain

LLM (for explanations)

Model:	Mistral 7B (via Ollama)
Framework:	Ollama (local inference)
Running On:	Local machine
Cost:	\$0 (open-source)
Speed:	Varies (CPU: 1-5 sec/response, GPU: 0.2-1 sec)

Why Ollama + Mistral:

- Completely free
- Run locally (no API costs)
- No rate limits
- Fast enough for MVP
- Good quality for explanations

RAG Framework

Library:	LangChain
Version:	Latest stable
Integration:	Supabase retriever + Ollama LLM

Why LangChain:

- Industry standard for RAG
- Great integrations
- Excellent documentation
- Production-ready

Development Tools

```
IDE:          VS Code
Version Control: Git + GitHub
Package Manager: npm (frontend), pip (backend)
Environment:  venv (Python virtual environment)
Runtime:      Node.js 16+ (frontend), Python 3.11+ (backend)
```

Data Sources & Pipeline

Recommended Datasets

Primary: Food.com Dataset

```
Source:      Kaggle
Size:        230K recipes
Quality:     ★★★★★
Includes:    Recipes, reviews, ratings, metadata
License:     Free/CC0
Use For:     Core recipe collection + vibe extraction from reviews
```

Secondary: RecipeNLG Dataset

```
Source:      Kaggle
Size:        2.2M recipes
Quality:     ★★★★☆
Includes:    Recipe names, ingredients, detailed instructions
License:     Free
Use For:     Scale & variety, better instruction parsing
```

Tertiary: Indian Recipes Dataset

Source:	Kaggle
Size:	10K+ recipes
Quality:	★★★★
Includes:	Cuisine-specific recipes with flavor profiles
License:	Free
Use For:	Cuisine diversity & regional flavor profiles

Optional: Spoonacular API (Real-time)

API:	https://spoonacular.com/food-api
Free Tier:	150 requests/day
Quality:	★★★★★
Includes:	High-quality recipes, nutrition data, detailed metadata
Use For:	Real-time data, advanced queries
Cost:	Free tier available

Data Pipeline Architecture

Step 1: Download Datasets

- └─ Food.com recipes.csv (230K)
- └─ RecipeNLG JSON (2.2M)
- └─ Indian recipes CSV (10K)

Step 2: Data Cleaning & Standardization

- └─ Parse ingredients
- └─ Normalize instructions
- └─ Extract metadata (cook time, difficulty)
- └─ Handle missing values

Step 3: Add Vibe Tags

- └─ Rule-based tagging from content
- └─ Extract vibes from reviews (Food.com)
- └─ Map cuisine to vibes
- └─ Normalize vibe categories

Step 4: Generate Embeddings

- └─ Create recipe embeddings (BAAI/bge-small)
- └─ Embed vibe descriptions
- └─ Store in 384-dim vectors
- └─ Process 2.5M recipes (~4-6 hours on CPU)

Step 5: Load into Supabase

- └─ Create pgvector indexes
- └─ Batch insert recipes
- └─ Create metadata indexes
- └─ Test vector search

Step 6: Quality Assurance

- └─ Verify embedding quality
- └─ Test vibe retrieval
- └─ Validate data integrity
- └─ Performance testing

Vibe Categories

Standard Vibes (12 categories):

1. COZY

- Keywords: warm, nostalgic, homemade, familiar, comfort
- Recipes: Chicken soup, pot roast, mac & cheese, grilled cheese
- Mood: Rainy day, feeling homesick, want comfort

2. ENERGETIC

- Keywords: quick, protein-rich, healthy, energizing
- Recipes: Breakfast bowls, smoothies, grain bowls, stir-fries
- Mood: Morning energy, gym pre/post-workout, active day

3. DATE-NIGHT

- Keywords: elegant, romantic, sophisticated, impressive
- Recipes: Pan-seared salmon, risotto, chocolate mousse, steak
- Mood: Special occasion, romantic dinner, impress someone

4. HEALTHY

- Keywords: salad, vegetables, light, low-calorie, nutritious
- Recipes: Buddha bowls, green salads, steamed fish, veggie medleys
- Mood: New Year's resolution, feeling virtuous, diet-conscious

5. ADVENTUROUS

- Keywords: exotic, spicy, international, unusual, bold
- Recipes: Thai curry, Indian biryani, Korean BBQ, Japanese ramen
- Mood: Exploring new cuisines, feeling bold, adventure seeker

6. INDULGENT

- Keywords: decadent, creamy, rich, chocolate, dessert
- Recipes: Tiramisu, brownies, cheesecake, creamy pasta

- Mood: Cheat day, celebrating, deserve a treat

7. FAMILY-FRIENDLY

- Keywords: kid-approved, simple, classic, mild, popular
- Recipes: Pizza, spaghetti, tacos, chicken nuggets
- Mood: Cooking for kids, need crowd-pleaser, quick & easy

8. QUICK-WEEKNIGHT

- Keywords: easy, fast (≤ 30 min), simple ingredients, minimal prep
- Recipes: Stir-fries, sheet pan meals, one-pot wonders
- Mood: Busy weekday, tired, minimal effort

9. WEEKEND-SPECIAL

- Keywords: impressive, time-intensive, detailed, showstopper
- Recipes: Homemade bread, slow-cooked stews, multi-step dishes
- Mood: Weekend has time, want to impress, project cooking

10. SEASONAL

- Keywords: fresh, seasonal, garden-fresh, harvest, berries
- Recipes: Summer salads, fall pumpkin dishes, winter roots
- Mood: Seasonal eating, farm-to-table, fresh ingredients

11. LIGHT & FRESH

- Keywords: Mediterranean, olive oil, fresh herbs, coastal
- Recipes: Greek salad, pasta primavera, grilled vegetables
- Mood: Summer feeling, Mediterranean vacation vibes

12. COMFORT-INDULGENT (Hybrid)

- Keywords: comforting, rich, hearty, satiating
- Recipes: Beef stew, lasagna, creamy soups
- Mood: Cold weather, want comfort that's also special

Data Statistics

Total Recipes:	2.5M+
Total Vibes:	12 primary + hybrid combinations
Average Ingredients:	8-10 per recipe
Instruction Length:	150-500 words average
Embedding Dimension:	384 (BAAI/bge-small)
Estimated Storage:	~1GB after embeddings (easily fits 500MB free tier)

System Architecture

High-Level Architecture Diagram



GET /recipes/{id}/explain
└─ Input: recipe ID, user vibe
└─ Output: RAG-generated explanation

GET /health
└─ Output: API status check

RAG Pipeline (LangChain)

1. EMBEDDING PHASE
└─ Convert user input to vector (BAAI/bge-small)
└─ Output: 384-dimensional vector
2. RETRIEVAL PHASE
└─ Query Supabase pgvector index
└─ Find top 10 similar recipes (cosine similarity)
└─ Output: Recipe objects with metadata
3. FILTERING PHASE
└─ Apply constraints (inventory, time, cuisine)
└─ Rank by relevance score
└─ Output: Top 5 filtered recipes
4. GENERATION PHASE
└─ Create prompt with retrieved recipes
└─ Send to Ollama (Mistral 7B)
└─ Generate personalized explanation
└─ Output: Natural language explanation
5. RESPONSE PHASE
└─ Format & return JSON to frontend
└─ Include recipes + explanation + metadata

Local Dev: <http://localhost:8000> (Uvicorn)
API Docs: <http://localhost:8000/docs> (Auto-generated Swagger)

↓ Vector Search ↓ LLM Generation

Supabase	Ollama
(PostgreSQL +	(Mistral 7B)

pgvector)		Local Inference	
500MB free tier		No API costs	
2.5M recipes		No rate limits	
384-dim vectors		runs on localhost	
		:11434	

Request Flow Example

User Input: "I'm feeling adventurous and want something spicy with the shrimp and ginger I have in my fridge"

FRONTEND:

- └ Parse input: vibe="adventurous spicy", ingredients=["shrimp", "ginger"]
- └ Send POST request to /recipes/combined
- └ Display loading state

BACKEND:

- └ Embedding:
 - └ Query text: "adventurous spicy shrimp ginger Asian cuisine"
 - └ Model: BAAI/bge-small-en-v1.5
 - └ Result: [0.234, -0.123, 0.456, ...] (384 dimensions)

- └ Retrieval:
 - └ Supabase query: Vector similarity search (cosine)
 - └ SQL: SELECT * FROM recipes
 - └ ORDER BY 1 - (vibe_embedding <=> query_vector)
 - └ LIMIT 10
 - └ Result: Top 10 recipes (Thai curry, Chinese stir-fry, etc)

- └ Filtering:
 - └ Check ingredients: Do recipes contain shrimp + ginger?
 - └ Check constraints: Cook time, difficulty, cuisine
 - └ Result: Top 5 filtered recipes ranked by match

- └ Generation:
 - └ Create prompt:
 - "User wants: adventurous spicy Asian food
 - With: shrimp, ginger
 - Top recipes:
 - 1. Thai Green Curry with Shrimp
 - 2. Szechuan Shrimp Stir-fry
 - 3. Korean Spicy Shrimp

```
| |  
| |   Explain why each matches and suggest combinations"  
| |  
| |─ Call Ollama (running on localhost:11434)  
| |─ Model: Mistral 7B  
| |─ Result: "Your adventurous spirit will love Thai Green Curry..."  
|  
└─ Response:  
    │─ Recipes: [curry_obj, stirfry_obj, korean_obj]  
    │─ Explanation: "Your adventurous spirit..."  
    │─ Metadata: prep_time, difficulty, ratings  
    └─ Return as JSON to frontend  
  
FRONTEND:  
└─ Parse response  
└─ Display recipe cards with ratings  
└─ Show RAG-generated explanation  
└─ User can click for full recipe details
```

Database Schema

Supabase Schema (PostgreSQL + pgvector)

```
sql
```

```

-- Enable pgvector extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Main recipes table
CREATE TABLE recipes (
    id BIGSERIAL PRIMARY KEY,
    external_id TEXT UNIQUE,
    name TEXT NOT NULL,
    description TEXT,
    ingredients JSONB NOT NULL,
    instructions JSONB NOT NULL,
    vibes JSONB, -- Array of vibe tags: ["cozy", "comfort", "warm"]
    cuisine TEXT,
    cook_time_minutes INTEGER,
    difficulty TEXT, -- 'easy', 'medium', 'hard'
    servings INTEGER,
    source TEXT, -- 'food_com', 'recipe_nlg', 'indian_recipes'
    vibe_embedding vector(384), -- Main embedding for vibe matching
    name_embedding vector(384), -- Alternative embedding for name search
    rating FLOAT, -- User rating (optional)
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Create indexes for performance
CREATE INDEX idx_recipes_vibe_embedding ON recipes
    USING ivfflat (vibe_embedding vector_cosine_ops);

CREATE INDEX idx_recipes_name_embedding ON recipes
    USING ivfflat (name_embedding vector_cosine_ops);

CREATE INDEX idx_recipes_cuisine ON recipes(cuisine);
CREATE INDEX idx_recipes_cook_time ON recipes(cook_time_minutes);
CREATE INDEX idx_recipes_difficulty ON recipes(difficulty);
CREATE INDEX idx_recipes_vibes ON recipes USING GIN (vibes);

-- Users table (for future features)
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email TEXT UNIQUE NOT NULL,
    username TEXT UNIQUE,
    preferences JSONB, -- Favorite cuisines, diets, allergies
    created_at TIMESTAMP DEFAULT NOW()
);

```

```

);

-- Favorites table (for future features)
CREATE TABLE favorites (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  recipe_id BIGINT REFERENCES recipes(id) ON DELETE CASCADE,
  created_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(user_id, recipe_id)
);

-- Search history (for analytics/learning)
CREATE TABLE search_history (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  vibe_query TEXT,
  ingredients TEXT[],
  recipe_results BIGINT[],
  clicked_recipe_id BIGINT,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Vector search helper function
CREATE OR REPLACE FUNCTION match_recipes(
  query_embedding vector(384),
  match_count INT DEFAULT 10,
  cuisine_filter TEXT DEFAULT NULL
)
RETURNS TABLE (
  id BIGINT,
  name TEXT,
  description TEXT,
  ingredients JSONB,
  vibes JSONB,
  cook_time_minutes INTEGER,
  difficulty TEXT,
  similarity FLOAT
) AS $$
BEGIN
  RETURN QUERY
  SELECT
    recipes.id,
    recipes.name,
    recipes.description,

```

```
    recipes.ingredients,  
    recipes.vibes,  
    recipes.cook_time_minutes,  
    recipes.difficulty,  
    1 - (recipes.vibe_embedding <=> query_embedding) AS similarity  
FROM recipes  
WHERE (cuisine_filter IS NULL OR recipes.cuisine = cuisine_filter)  
ORDER BY recipes.vibe_embedding <=> query_embedding  
LIMIT match_count;  
END;  
$$ LANGUAGE plpgsql;
```

Data Examples

json

```
{
  "id": 1001,
  "external_id": "foodcom_12345",
  "name": "Thai Green Curry with Shrimp",
  "description": "Authentic Thai green curry with succulent shrimp, fresh basil, and",
  "ingredients": [
    {"name": "shrimp", "amount": 1.5, "unit": "lbs"},
    {"name": "coconut milk", "amount": 2, "unit": "cans"},
    {"name": "green curry paste", "amount": 3, "unit": "tbsp"},
    {"name": "fresh basil", "amount": 1, "unit": "cup"},
    {"name": "lime", "amount": 2},
    {"name": "ginger", "amount": "2 inches"},
    {"name": "fish sauce", "amount": 2, "unit": "tbsp"}
  ],
  "instructions": [
    "Heat coconut milk in a large pan over medium-high heat",
    "Add green curry paste and stir until fragrant",
    "Add shrimp and cook until pink (5-7 minutes)",
    "Add vegetables and fresh basil",
    "Finish with lime juice and serve over rice"
  ],
  "vibes": ["adventurous", "spicy", "exotic", "quick-weeknight", "healthy"],
  "cuisine": "thai",
  "cook_time_minutes": 20,
  "difficulty": "medium",
  "servings": 4,
  "source": "food_com",
  "vibe_embedding": [0.234, -0.123, 0.456, ...], // 384 dimensions
  "rating": 4.5,
  "created_at": "2024-01-15T10:30:00Z"
}
```

Project Structure

Local macOS Setup

```
recipe-vibe-app/
|
├─ frontend/           # React + Vite app
|  └─ src/
```

```

| | | └─ components/
| | | | └─ MoodSelector.jsx      # Vibe input UI
| | | | └─ InventoryInput.jsx   # Ingredient checklist
| | | | └─ RecipeCard.jsx       # Recipe display card
| | | | └─ RecipeList.jsx       # Results grid
| | | | └─ LoadingState.jsx     # Skeleton/loader
| | | └─ pages/
| | | | └─ Home.jsx             # Main page
| | | | └─ RecipeDetail.jsx     # Full recipe view
| | | | └─ Favorites.jsx        # Saved recipes (future)
| | | └─ hooks/
| | | | └─ useRecipeSearch.js   # API calls
| | | | └─ useVibeMatcher.js    # Vibe logic
| | | └─ services/
| | | | └─ api.js               # API client
| | | └─ styles/
| | | | └─ globals.css          # Tailwind import
| | | └─ App.jsx               # Main component
| | | └─ main.jsx              # Entry point
| | └─ public/                 # Static assets
| | └─ package.json            # Dependencies
| | └─ vite.config.js          # Vite configuration
| | └─ tailwind.config.js      # Tailwind config
|
└─ backend/                   # FastAPI + Python
|   └─ app/
|       └─ main.py             # FastAPI app setup
|       └─ routes/
|           └─ recipes.py       # Recipe endpoints
|           └─ health.py        # Health check
|       └─ services/
|           └─ rag_pipeline.py  # LangChain RAG logic
|           └─ embeddings.py    # Embedding service
|           └─ llm_service.py   # Ollama integration
|           └─ supabase_client.py # DB operations
|       └─ models/
|           └─ recipe.py        # Recipe data model
|           └─ search.py        # Search request model
|       └─ utils/
|           └─ config.py        # Environment variables
|           └─ logger.py        # Logging setup
|   └─ data_pipeline/          # Data processing scripts
|       └─ download_datasets.py # Kaggle downloads
|       └─ process_data.py      # Data cleaning

```

```

|   |   |─ add_vibes.py           # Vibe tagging
|   |   |─ generate_embeddings.py  # Embedding generation
|   |   |─ supabase_loader.py     # Load to database
|   |─ requirements.txt           # Python dependencies
|   |─ .env.example              # Environment template
|   |─ venv/                     # Python virtual environment
|
|─ docs/                         # Documentation
|   |─ PROJECT_SCOPE.md          # This file
|   |─ IMPLEMENTATION_GUIDE.md   # Opus 4.7 reference
|   |─ API_REFERENCE.md          # Endpoint documentation
|   |─ SETUP_GUIDE.md           # Local setup steps
|
|─ .gitignore                   # Git ignore rules
|─ .env                         # Local environment (git ignored)
|─ README.md                    # Project overview
|─ docker-compose.yml           # Optional: Docker setup

```

File-by-File Breakdown

FRONTEND FILES (React):

App.jsx (150 lines)

└─ Main component, routing, state management

components/

└─ MoodSelector.jsx (100 lines)

| └─ UI for mood selection (buttons/text input)

└─ InventoryInput.jsx (150 lines)

| └─ Checklist for ingredients

└─ RecipeCard.jsx (120 lines)

| └─ Single recipe display

└─ RecipeList.jsx (100 lines)

| └─ Grid of recipe cards

└─ LoadingState.jsx (50 lines)

| └─ Loading skeleton

pages/

└─ Home.jsx (200 lines)

| └─ Main search page

└─ RecipeDetail.jsx (150 lines)

| └─ Full recipe view

hooks/

- └─ useRecipeSearch.js (100 lines)
 - └─ API call logic
- └─ useVibeMatcher.js (80 lines)
 - └─ Vibe logic

services/

- └─ api.js (80 lines)
 - └─ Axios/fetch client setup

BACKEND FILES (Python):

app/main.py (100 lines)

- └─ FastAPI initialization, middleware

app/routes/

- └─ recipes.py (200 lines)
 - └─ POST /recipes/by-vibe
 - └─ POST /recipes/by-inventory
 - └─ POST /recipes/combined
 - └─ GET /recipes/{id}
 - └─ GET /recipes/{id}/explain
- └─ health.py (30 lines)
 - └─ GET /health

app/services/

- └─ rag_pipeline.py (300 lines)
 - └─ LangChain RAG chain
 - └─ Retrieval logic
 - └─ Generation logic
- └─ embeddings.py (80 lines)
 - └─ Sentence-Transformers wrapper
- └─ llm_service.py (60 lines)
 - └─ Ollama integration
- └─ supabase_client.py (100 lines)
 - └─ Database operations

app/models/

- └─ recipe.py (50 lines)
 - └─ Pydantic models
- └─ search.py (50 lines)
 - └─ Request/response models

```
app/utils/  
├─ config.py (30 lines)  
│   └─ Environment setup  
└─ logger.py (20 lines)  
    └─ Logging config
```

DATA PIPELINE FILES (Python):

```
download_datasets.py (100 lines)  
└─ Kaggle API integration
```

```
process_data.py (300 lines)  
├─ Data cleaning  
├─ Standardization  
└─ Combination logic
```

```
add_vibes.py (150 lines)  
└─ Vibe tag assignment
```

```
generate_embeddings.py (200 lines)  
├─ Batch embedding generation  
├─ Progress tracking  
└─ Error handling
```

```
supabase_loader.py (150 lines)  
├─ Batch insertion  
└─ Index creation
```

TOTAL CODE ESTIMATE:

```
Frontend:    ~1,500 lines JavaScript  
Backend:     ~1,200 lines Python  
Data Pipeline: ~900 lines Python
```

```
TOTAL:       ~3,600 lines
```

Feature Specifications

Core Features (MVP)

1. Vibe-Based Recipe Search

User Input:

Text: "I'm feeling cozy and want warm comfort food"

or

Mood Button: [Cozy] [Warm] [Comfort]

Processing:

1. Parse/embed user input

2. Vector search in Supabase

3. Retrieve top 10 recipes

4. Filter by vibe match confidence

5. Generate explanation with Ollama

Output:

json

{

"recipes": [

{

"id": 1001,

"name": "Homemade Chicken Soup",

"cook_time": 45,

"vibes": ["cozy", "comfort", "warm"],

"match_score": 0.92

}

],

"explanation": "This classic chicken soup perfectly captures that warm, homemade c

}

2. Inventory-Based Filtering

User Input:

Ingredients: ["chicken", "garlic", "olive oil", "tomatoes"]

Processing:

1. Get all recipes with these ingredients
2. Rank by match percentage
3. Sort by cook time (optional)

Output:

```
json
{
  "recipes": [
    {
      "name": "Chicken Tomato Pasta",
      "ingredient_match": 95,
      "missing_items": ["pasta", "parmesan"]
    }
  ]
}
```

3. Combined Search

User Input:

```
{
  "vibe": "quick weeknight dinner",
  "ingredients": ["chicken", "rice"],
  "max_cook_time": 30,
  "cuisine": "asian"
}
```

Processing:

1. Embed vibe
2. Vector search
3. Filter by ingredients
4. Filter by constraints
5. Rank and return top 5

4. Recipe Details Page

Display:

- Full ingredient list with quantities
- Step-by-step instructions
- Cook time & difficulty
- Serving size
- Nutritional info (if available)
- Similar recipes
- Save/favorite button (future)

5. RAG-Generated Explanations

Example:

```
User: "I want something adventurous"
System Retrieved: Thai Green Curry
Explanation: "Thai Green Curry is perfect for your adventurous mood!
It features bold, complex flavors - the green curry paste brings heat
and depth, while fresh basil and coconut milk create a creamy contrast.
The shrimp cooks quickly, making it ideal for a weeknight adventure."
```

Data Models

```
python
```

```

# Pydantic models for type safety

class RecipeBase(BaseModel):
    name: str
    description: str
    ingredients: List[Dict[str, str]]
    instructions: List[str]
    vibes: List[str]
    cuisine: str
    cook_time_minutes: int
    difficulty: str

class RecipeCreate(RecipeBase):
    pass

class Recipe(RecipeBase):
    id: int
    external_id: str
    source: str
    rating: Optional[float] = None
    created_at: datetime

class RecipeSearchRequest(BaseModel):
    vibe: str
    ingredients: Optional[List[str]] = None
    max_cook_time: Optional[int] = None
    cuisine: Optional[str] = None
    max_results: int = 5

class RecipeSearchResponse(BaseModel):
    recipes: List[Recipe]
    explanation: str
    total_matches: int
    search_time_ms: float

```

Implementation Phases

Phase 1: MVP (Weeks 1-3)

Goal: Basic vibe-based recipe discovery

Tasks:

1. ☒ Set up project structure (Git, venv, npm)
2. ☒ Download & prepare datasets
3. ☒ Build data pipeline (cleaning, vibes, embeddings)
4. ☒ Set up Supabase with pgvector
5. ☒ Create FastAPI backend with RAG
6. ☒ Build React frontend with Vite
7. ☒ Integrate Ollama for explanations
8. ☒ Test end-to-end flow
9. ☒ Deploy to Railway + Vercel

Deliverables:

- 2.5M recipes indexed and searchable
- Working vibe-based search
- Simple but functional UI
- RAG explanations working

Phase 2: Enhancement (Weeks 4-5)

Goal: Better UX and features

Tasks:

1. Add inventory-based filtering
2. Improve UI/UX with DaisyUI components
3. Add recipe detail pages
4. Implement favorites system
5. Add user accounts (optional)
6. Better error handling
7. Performance optimization

Phase 3: Advanced Features (Weeks 6+)

Goal: Production-ready platform

Tasks:

1. Meal planning
2. Shopping list generation

- 3. Nutritional tracking
- 4. Social features
- 5. Mobile app (React Native)
- 6. Advanced analytics

Cost Analysis

Development Phase (Months 1-3)

Service	Free Tier	Cost
Vercel (Frontend)	Unlimited	\$0
Railway (Backend)	\$5/mo credit	\$0
Supabase (Database)	500MB	\$0
Sentence-Transformers	Open-source	\$0
Ollama	Open-source	\$0
LangChain	Open-source	\$0
Kaggle Datasets	Free	\$0
GitHub	Free tier	\$0
Domain (optional)	—	\$10-15/yr
TOTAL		\$0/month

Post-MVP Costs (Production)

Estimated after free credits exhaust:

Service	Usage	Cost
Vercel	Standard	\$0 (generous free tier)
Railway	FastAPI + PostgreSQL	\$10-20/mo
Supabase	Larger database (>500MB)	\$5-15/mo
Embeddings	Local (no cost)	\$0
LLM	Local Ollama (no cost)	\$0
Domain	Custom domain	\$1-15/yr
TOTAL		~\$15-40/month

Very affordable compared to competitors using paid APIs (\$100-500/mo)

Deployment Guide

Local Development Setup (macOS)

```
bash
```

```
# 1. Clone or create repository
mkdir recipe-vibe-app
cd recipe-vibe-app
git init

# 2. Frontend setup
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install -D tailwindcss daisyui postcss autoprefixer
npm run dev # Runs on http://localhost:5173

# 3. Backend setup
cd ..
python3 -m venv venv
source venv/bin/activate
pip install fastapi uvicorn langchain supabase-py sentence-transformers

# 4. Ollama setup (separate terminal)
# Download from ollama.ai
ollama pull mistral
ollama serve # Runs on http://localhost:11434

# 5. Supabase setup
# Create account at supabase.com
# Create project → Get URL + API key

# 6. Run backend
cd backend
python main.py # Runs on http://localhost:8000
```

Production Deployment

Frontend to Vercel:

```
bash

cd frontend
npm run build
# Connect GitHub repo to Vercel → Auto-deploys on git push
```

Backend to Railway:

1. Connect GitHub repo to Railway
2. Add environment variables:
 - SUPABASE_URL
 - SUPABASE_KEY
 - OLLAMA_URL=http://localhost:11434
3. Railway auto-detects Python + deploys

API Reference

Endpoints

POST /recipes/by-vibe

└─ Request: { "vibe": "cozy comfort food" }
└─ Response: { recipes: [...], explanation: "..." }

POST /recipes/by-inventory

└─ Request: { "ingredients": ["chicken", "tomato"] }
└─ Response: { recipes: [...], matching: 85 }

POST /recipes/combined

└─ Request: {
 | "vibe": "quick weeknight",
 | "ingredients": ["chicken"],
 | "max_cook_time": 30,
 | "cuisine": "asian"
 | }
└─ Response: { recipes: [...], explanation: "..." }

GET /recipes/{id}

└─ Request: /recipes/1001
└─ Response: { full recipe object }

GET /recipes/{id}/explain

└─ Request: /recipes/1001/explain?vibe="adventurous"
└─ Response: { explanation: "RAG-generated text" }

GET /health

└─ Request: /health
└─ Response: { status: "ok", services: {...} }

Environment Variables

Required (.env file)

```
bash

# Supabase
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=your-anon-key

# Ollama
OLLAMA_BASE_URL=http://localhost:11434

# FastAPI
FASTAPI_ENV=development
LOG_LEVEL=INFO

# Frontend
VITE_API_URL=http://localhost:8000
```

Testing Strategy

Unit Tests

```
python

# Test embedding generation
# Test RAG pipeline
# Test database queries
# Test API endpoints
```

Integration Tests

```
python

# Test full request flow
# Test RAG end-to-end
# Test Supabase integration
```

Manual Testing

1. Test vibe-based search with various inputs
 2. Test inventory filtering
 3. Test combined constraints
 4. Test edge cases (empty results, partial matches)
 5. Test UI responsiveness on different devices
-

Monitoring & Logging

What to Monitor

- API response times
- Embedding generation time
- Vector search performance
- LLM generation time
- Error rates
- User searches (for analytics)

Tools

- Console logging (built-in)
 - FastAPI logging
 - React error boundaries
 - Supabase logs
-

Future Enhancements

Short Term (Month 2-3)

- User accounts & authentication
- Save favorite recipes
- Recipe history
- Better UI/UX polish
- Mobile-responsive design

Medium Term (Month 4-6)

- Meal planning (7-day plans)
- Shopping list generation
- Nutritional tracking
- Recipe ratings & reviews
- Social sharing

Long Term (Month 7+)

- Mobile app (React Native)
 - Advanced search filters
 - Cuisine fusion explorer
 - Cooking difficulty progression
 - Restaurant partnerships
 - Ingredient price tracking
-

Troubleshooting

Common Issues

Ollama not responding

```
Solution: Ensure Ollama is running
$ ollama serve
Check: http://localhost:11434/api/tags
```

Embeddings slow

```
Solution: Use GPU acceleration
- NVIDIA GPU with CUDA installed
- M1/M2 Mac with Metal support
- Can reduce time from 6hrs to 30min for 2.5M recipes
```

Supabase vector search slow

```
Solution: Ensure indexes created
- Check pgvector index exists
- Increase work_mem in PostgreSQL settings
```

Frontend can't reach backend

```
Solution: Check CORS and ports
- Backend on :8000
- Frontend on :5173
- FastAPI CORS middleware configured
```

References & Resources

Documentation

- FastAPI: <https://fastapi.tiangolo.com/>
- LangChain: <https://js.langchain.com/>
- Supabase: <https://supabase.com/docs>
- Ollama: <https://ollama.ai>
- Sentence-Transformers: <https://www.sbert.net/>
- React: <https://react.dev/>
- Tailwind CSS: <https://tailwindcss.com/>

Tools

- VS Code: Code editor
- Git: Version control
- Vercel: Frontend hosting
- Railway: Backend hosting
- Supabase: Database hosting

Contact & Support

For questions or issues:

- Check documentation first

- Review troubleshooting section
 - Check API logs for errors
 - Verify environment variables
-

Document Version: 1.0

Last Updated: January 2025

Status: Reference Document